

SmartCare v0.4 - Domain Implementation Workbook

Week 7 student resource

1. UML-to-Code Trace

UML element	Python element	Implemented?	Notes
Patient (class)	class Patient	Yes	Type hints, no database logic
Patient: patient_id, name	Private attributes with read-only properties	Yes	Name validated, never blank
Patient.matches(query)	Method matches()	Yes	Matches by ID or name
Practitioner (class)	class Practitioner	Yes	No database logic
Practitioner: practitioner_id, name, specialty	Private attributes with read-only properties	Yes	Specialty comes from the lab spec and is not in the v0.3 UML (see Section 5)
Practitioner.appointments_on(date)	Method appointments_on()	Skeleton only	Needs appointments to be stored somewhere, which is not built yet
Appointment (class)	class Appointment	Yes	Based on the ChatGPT output, refactored in Part G
Appointment: patient, practitioner, date, time	Private attributes	Yes	Private, read-only properties
Appointment: status	AppointmentStatus enum	Yes	Enum with SCHEDULED, CANCELLED, COMPLETED
Appointment.cancel(), conflicts_with()	Methods cancel() and conflicts_with()	Yes	Illegal status changes raise InvalidAppointmentStatusError

2. Domain Invariants

Class	Invariant / rule	How protected
Patient	patient_id is unique and never changes	Private attribute with a read-only property and no setter
Patient	name is never blank	Validated in the constructor and in rename()
Practitioner	ID never changes; name and specialty are never blank	Private attributes validated in the constructor
Appointment	Patient, practitioner, date and time are fixed after creation	Private attributes with no setters
Appointment	Status changes only through allowed transitions	Status is private; cancel() and complete() check the current status
Appointment	A cancelled or completed appointment cannot be cancelled again	Raises the agreed domain exception

3. Composition / Inheritance Decisions

Relationship	Decision	Rationale
Appointment and Patient	Association (has-a)	An appointment refers to a patient but is not a kind of patient
Appointment and Practitioner	Association (has-a)	An appointment refers to a practitioner but is not a kind of practitioner
Doctor and Practitioner (hypothetical)	Not implemented	Inheritance only makes sense if doctors need different behaviour, and nothing requires that yet
AppointmentBook and Appointment	Association (holds a list)	The optional AppointmentBook holds appointments, which exist as their own objects
Appointment and PatientRecord / NotificationManager (AI-style suggestions)	Rejected	Not in the approved UML. An appointment is not a patient record, and notifications were never requested

4. AI Pair-Programming Record

AI contribution	Conforms?	Decision	Reason	Verification
AppointmentStatus enum and InvalidAppointmentStatusError	Yes	Accepted	Matches the agreed statuses and the one agreed exception	Checks: new appointment starts SCHEDULED; second cancel raises the exception
Private status changed only by cancel() and complete()	Yes	Accepted	Protects the status invariant and avoids public mutation	Checks: setting status directly fails; cancel works on a scheduled appointment
Date and time type checks using __import__("datetime")	Partly	Modified	It worked but was unclear because the parameter names clash with the imported types	Replaced with import datetime and datetime.date hints; re-ran checks and a text date is still rejected
Duplicate logic in cancel() and complete()	Yes	Modified	Both repeated the same status check, so I moved it into one shared method	Re-ran checks: cancel, complete and the repeated cancel still behave the same
conflicts_with(): same practitioner, date and time, ignoring cancelled appointments	Yes	Accepted	Matches the provisional double-booking rule, and the AI stated its assumption	Checks: conflict is True, then False after cancelling

Prompt used for ChatGPT: Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

5. Updated UML

Insert updated UML only if implementation revealed a justified design change. Explain every change.

Changes: add specialty to Practitioner (required by the lab spec), add an AppointmentStatus enum, and add a domain exception for illegal status changes. No other changes. The design otherwise matches the v0.3 UML.

Reflection:

I modified the date and time type checks, because the AI used an `__import__("datetime")` workaround that was confusing and unnecessary. I replaced it with a normal `import datetime`. I also merged the repeated status check in `cancel()` and `complete()` into one method. I accepted the Enum, the exception and the private status, because they matched the design.

The approved design kept the AI on track. It added no database, UI, notification or inheritance. It also stated it's one assumption about double-bookings, so I could check it against the design.